LEÇON : UTILISATION DES FEUILLES DE STYLE ET ORGANISATION DU DOSSIER DU SITE

Situation problème

Compétences visées :

- ***Définir et donner les avantages d'une feuille de style***
- ***Utiliser une feuille de style dans un document html***
- ***Organiser le dossier d'un site web***

I. DEFINITION ET AVANTAGES D'UNE FEUILLE DE STYLE

- **Le CSS (Cascading Style Sheet)** est un langage permettant la mise en forme des pages Web. Il sur des éléments de langage html nettement sur des balises qui représentent des sélecteurs.
- **Une feuille de style** est une portion d'un document html qui permet la mise en forme totale ou partielle du document.

Les avantages d'une feuille de style sont :

- Séparation du contenu et de la mise en forme.
- Cohésion de la présentation tout au long du site avec les feuilles de style externes.
- Modifier l'aspect d'une page ou d'un site sans en modifier le contenu
- Réduire le temps de chargement des pages.
- Palier certaines insuffisances du langage Html (contrôle des polices, contrôle de la distance entre les lignes) et ainsi augmenter la créativité des écrivains du Web.

II. UTILISATION DES FEUILLES DE STYLE

Dans un document html, on peut insérer le code css soit :

1. Dans une balise html sous forme d'attribut

Syntaxe : <balise style="propriété : valeur">
Exemple: <p style="color :blue">

2. Dans l'entête de document html : entre les balises <head>……</head>

On insère la balise <style type="text/css" > Saisir le code css ici </style>

3. Dans un fichier de style externe d'extension .css

Dans les deux derniers cas, on utilise la syntaxe générale du css qui est la suivante :
Sélecteur { Propriété : valeur ;}
Exemple p { font-size:12px; color :blue ; }
Pour faire agir le css sur le html, il faut ajouter à l'entête du fichier html la balise
<link rel="stylesheet" href="chemin/fichier.css" type="text/css">
L'attribut href permet d'indiquer l'emplacement du fichier css dans l'ordinateur

III. ORGANISER LE DOSSIER DU SITE

Dans la mesure du possible, de mettre le dossier principal du site 3 éléments :

- **Le fichier index.html** : le nom à donner à votre page d'accueil sera imposé par votre fournisseur d'accès. Le plus souvent ce sera « index.htm » ou « index.html ». Il s'agit de la première page de votre site que l'utilisateur verra. C'est la seule page dont le fournisseur d'accès a besoin de connaître le nom. Les autres pages ainsi que les images sont organisées selon votre souhait.
- Un sous-dossier nommé par exemple « pages » : qui contiendra les autres pages de votre site
- Un sous-dossier nommé par exemple images : qui contiendra les images de votre site

L'utilisation des informations enregistrées implique l'utilisation des liens hypertexte. La balise <a> permet d'insérer un lien hypertexte

CHAPITRE 4: PROGRAMMER EN JAVASCRIPT

LEÇON : INTRODUCTION AU JAVASCRIPT

Compétences visées :

- ***Énoncer et expliquer les limites du HTML liées à l'interactivité***
- ***Énoncer et décrire les avantages et les limites de Javascript ;***
- ***Utiliser la balise <script>…</script> ;***
- ***Décrire et utiliser les opérateurs en JavaScript***

I. GENERALITES

1. Définitions

JavaScript est un langage de scripts orienté-objet qui peut être incorporé dans un document html pour apporter plus d'interactivité aux pages web.

Un script est un ensemble d'instructions intégrer dans le code d'une page html et interpréter par le navigateur.

NB :

- Le Javascript est un langage client-side (coté client) car il est interprété et exécuté par un navigateur.
- Le JavaScript est un langage interprété c.a.d la traduction de son code source se fait en temps réel lors de l'exécution

2. Les limites du HTML liées à l'interactivité

L'interactivité correspond au changement observable suite à une action (le click ou le déplacement de la souris, la saisie de données dans une zone de texte, etc.) effectuée par l'utilisateur dans la page web. Les limites du html liées à l'interactivité peuvent être :

- Le html ne permet pas d'afficher les boîtes de dialogue
- Le html ne permet pas de soumettre les informations saisies dans le formulaire
- Le html ne permet pas le contrôle des données saisies dans le formulaire
- Le html ne permet pas d'effectuer les calculs
- Le html ne prend pas en compte la gestion des évènements tels que le click ou le déplacement de la souris

II. AVANTAGES ET LIMITES DU JAVASCRIPT

1. Avantages du JavaScript

L'avantage principal du JavaScript est qu'il permet de rendre les pages web interactives ou dynamiques. En effet, le JavaScript est important car il permet :

- La vérification des champs du formulaire ;

- La validation du formulaire ;
- L'affichage des messages d'alerte ;
- La gestion des évènements tels que le click ou le déplacement de la souris ;
- La récupération des valeurs entrées par l'utilisateur pour exécuter des commandes ;
- La réalisation des opérations ;

Par ailleurs, le JavaScript contient aussi des limites.

2. Limites du JavaScript

Comme limites, nous avons :

- JavaScript n'est pas compatible à tous les navigateurs web ;
- Le code JavaScript n'est sécurisé. Il est visible par tout le monde ;
- JavaScript ne permet d'écrire ou de lire sur le disque dur ;
- Les erreurs en JavaScript sont difficiles à détecter à cause de l'absence d'un débuggeur ;
- JavaScript ne permet pas de faire des bases de données ;
- JavaScript ne permet pas de réaliser des forums de discussion.

III. INSERTION DU CODE JAVASCRIPT DANS LA PAGE HTML

Tout script JavaScript est délimité par les balises <script> et </script>. La syntaxe générale du code JavaScript est définie par l'une des deux méthodes suivantes :

Méthode 1 :
<script language="**javascript**">
Ici le code javascript ;
</script>

Méthode 2 :
<script type="**text/javascript**">
Ici le code javascript ;
</script>

Le code JavaScript peut être inséré dans 03 endroits du document HTML :

- Dans l'en-tête du document html entre les balises <head>…</head>
- Dans une balise situé dans le corps du document html entre les balises <body>…</body> (méthode peu recommandée car rend la page web lourde)
- Dans un fichier externe portant l'extension .js. Dans ce cas, il faut écrire dans l'en-tête du code html le lien vers le fichier. Par exemple :
  <script type="text/javascript" src="MonFichier.js"></script>

Exemple de code javascript : Ecrire le code JavaScript qui permet d'afficher le message « Bonne et heureuse année scolaire à tous les élèves. » en utilisant la méthode **document.write()**.

```
<html>
<head><title> Voici une page contenant du Javascript </title></head>
<body>
<script language="Javascript">
  document.write ("Bonne et heureuse année scolaire à tous les élève");//Permet d'afficher le message.
</script>
</body>
</html>
```


NB : la méthode document.write permet d'afficher le contenu dans une page

IV. LES OPERATEURS EN JAVASCRIPT

On distingue :

1. Les opérateurs arithmétiques

Opérateur	Description
+	Addition
-	Soustraction
*	Multiplication
/	Division
^	Puissance
%	Modulo (a%b renvoie le reste de la division entière de a par b)
+ +	Incrémentation // x++ équivaut à x = x +1
--	Décrémentation // x-- équivaut à x = x - 1

2. Les opérateurs logiques

Opérateur	Description
&&	ET logique
||	OU logique
!	Négation logique

3. Operateurs de comparaisons

Opérateur	Description
==	Est égal à
!=	Différent de
>	Strictement supérieur à
<	Strictement inférieur à
>=	Supérieur ou égal
<=	Inférieur ou égal

4. L'opérateur de chaine ou de concaténation

L'opération entre les chaines de caractère s'appelle la concaténation, c'est-à-dire la juxtaposition des variables de type chaine. L'opérateur de concaténation est **'+'.**

Exemples:
Soient les variables suivantes **:**
a=2 ; b=8 ; c=5 ; texte1= "bonjour " ; texte2= "le monde ! " ;

Instruction	Résultat
c = b – c ;	c = 3
d = (a > 0) && (c < b) ;	True (1)

alert (texte1 + texte2) ;	Bonjour tout le monde
a++ ;	a = 3
b-- ;	b = 7
(a + c) == (b – c % 2);	False (0)

Remarque : L'expression **variable = variable operateur constante** (**a = a + 3** par exemple) peut être simplifiée en **variable opérateur= constante** (**a += 3**, qui lui est équivalente). Ainsi, au lieu d'écrire **nombre = nombre * 26**, on peut écrire simplement **nombre*=26** ; il en est de même pour tous les opérateurs arithmétiques.

LEÇON : LES BASES DU JAVASCRIPT

Compétences visées :

- ***Déclarer les variables en JavaScript***
- ***Utiliser les instructions de base en JavaScript***
- ***Effectuer les conversions de type***
- ***Utiliser l'événement OnClick***

I. LA NOTION DE VARIABLE EN JAVASCRIPT

1. Définition et nomenclature d'une variable

Une variable est un objet mémoire dont le contenu peut être modifié au cours de l'exécution du script. En JavaScript, le nom d'une variable doit respecter les règles suivantes :

- Il doit commencer par une lettre majuscule, minuscule ou par le caractère **underscore**(**_**) ;
- Il ne doit pas commencer par un chiffre ;
- Il ne doit pas contenir des espaces ;
- Il ne doit pas être un mot réservé comme : **Boolean, char, int, public, if, for, return, while, float, private, function,** etc.

En outre, javascript est sensible à la casse ! C'est-à-dire que javascript fait la différence entre les majuscules et les minuscules, les caractères accentuées et les caractères non-accentués.

Exemples :« **Nom** » est différente de « **nom** » de même que « **élève** » est différent de « **eleve** »

2. Déclaration d'une variable

En JavaScript, il existe deux types de déclarations de variables : la déclaration **explicite** et la déclaration **implicite**.

- Pour déclarer explicitement une variable, il faut utiliser le **mot clé « var »**. Il permet d'indiquer de façon rigoureuse que c'est une variable.

Exemple :

Var x ; // déclaration explicite

Var x=18 ; //déclaration explicite avec affectation de valeur.

- Pour déclarer une variable de manière implicite, le mot-clé « var » n'est pas utilisé. Il suffit d'écrire le nom de la variable suivie du symbole **=** (égal) et de la valeur à affecter.

Exemple : **chaine="bonjour"**. Dans la déclaration implicite, le soin est laissé au navigateur de déterminer qu'il s'agit d'une déclaration de variable

NB **: Même si une déclaration implicite est tout à fait reconnue par le navigateur, il est plus rigoureux de déclarer les variables de façon explicite avec le mot-clé «** var **».**

3. Les types de données en JavaScript

Un type de données correspond à un ensemble de valeurs qu'une variable peut prendre. En JavaScript, il n'est pas nécessaire de déclarer le type de variable, une variable prend implicitement le type de la valeur qui lui est attribué (raison pour laquelle on dit que le JavaScript est très **peu typé** ou **faiblement typé**). JavaScript utilise quatre types de données :

- **Number** : il s'agit des nombres entiers ou à virgule tels que **22** ou **3.14** ;
- **String** : il s'agit des chaines de caractères tels que **"bonjour"** ;
- **Boolean** : il s'agit des variables qui peuvent prendre deux valeurs : « **True »** si le résultat est vrai ou « **False** » si le résultat est faux. Elles permettent de vérifier une condition ;
- **Null** : il s'agit des variables qui ne contiennent aucune donnée.

II. LES INSTRUCTIONS DE BASES EN JAVASCRIPT

Une instruction est l'opération élémentaire que le processeur peut accomplie en une seule fois. Nous nous intéresserons ici aux instructions de base tels que : l'instruction d'affectation, les instructions d'entrées ou de lecture, les instructions de sorties ou d'affichage et enfin l'instruction de confirmation

1. L'instruction d'affectation

Cette instruction permet d'attribuer une valeur ou le résultat d'une expression à une variable.

Syntaxe : **nom_variable=valeur** ou **nom_variable= expression à calculer**;

Exemple : nombre = 19 ; prenom = "zavier" ; produit = 2 * 15 ; surface = Lon * Lar ;

2. L'instruction de lecture ou d'entrée.

Cette instruction permet de récupérer les informations saisies par l'utilisateur et de les stocker dans les variables. Pour cela on utilisera la méthode **prompt().**

Syntaxe : **variable = prompt ("texte à afficher"**).

Exemple : Nom = **prompt** ("Entrer votre nom") ;

3. Les instructions d'affichage, de sortie ou d'écriture

Pour afficher du texte en JavaScript on peut utiliser la méthode **document.write()** qui affiche le message directement sur la page web ou la méthode **alert()** qui affiche le message dans une boite de dialogue

Syntaxe : **document.write**("texte à afficher") ; ou **alert**("texte à afficher") ;

document.write(nom_variable) ; ou **aler**t(nom_variable) ;

Exemple :**alert**("Que c'est facile la première scientifique !") ;

NB : Il est aussi possible de demander une confirmation pour une action (suppression d'élément, fermeture de page, …). On utilise la fonction **«confirm»** dont la syntaxe est **variable** = **confirm**("**Demande de confirmation**") ;

Exemple : confirmation = **confirm**("Voulez-vous fermer la page ?") ;

NB : les méthodes alert(), prompt() et confirm() sont présentées sous forme de boites de dialogue

III. LES CONVERSIONS DE TYPE

Par défaut, les variables en javascript sont de types chaine de caractères (String). Il sera souvent nécessaire de convertir ces variables en types entier ou en type réel.

- La fonction **parseInt(**) permet de convertir une variable en nombre entier.
- La fonction **parseFloat()** permet de convertir une variable en nombre réel.

Exemples	Résultats
parseInt("128.34 ")	128
parseInt("0284,8 ")	284
parseFloat("128,34") ;	128

IV. L'EVENEMENT ONCLICK

Les événements sont des actions de l'utilisateur qui vont pouvoir donner lieu à une interactivité. L'événement par excellence est le clic de souris. Il est possible d'associer des actions à des événements aux moyens des gestionnaires d'événements. Le nom d'un gestionnaire d'événement commence toujours par le mot « **on** » ; on peut par exemple citer : **onclick / ondbclick** : lorsqu'on clique ou double clique avec la souris

Les gestionnaires d'événements sont associés à des objets, et leur code s'insère dans la balise de ceux-ci ; la syntaxe d'utilisation d'un gestionnaire d'événement est la suivante :

onEvenement= "Action_Javascript ou Fonction();"

Exemple : **onclick**= "alert("Bonjour");"

ACTIVITE D'INTEGRATION

Soit le script : <**script language** = "javascript">
Var nombre1 = parseInt(prompt("entrer le premier nombre svp"));
Var nombre2 = prompt("entrer le deuxième nombre svp") ;
alert(nombre1 + nombre2**) ;**
</script>

1 ; Quel est le type de donnée retourné par défaut dans un prompt
2 Déduire le rôle du parseInt dans ce script
3 Quel sera le contenu de la boite de dialogue alert si l'utilisateur entre respectivement
   a) 4 et 88
   b) 23bonjour et salut
4 Corriger ce script pour retourner effectivement la somme de nombre1 et

nombre2

LEÇON : LES STRUCTURES ET LES FONCTIONS EN JAVASCRIPT

Compétences visées :

- ***Traduire des algorithmes séquentiels, conditionnels et itératifs en JavaScript***
- ***Déclarer une fonction en JavaScript***

I. LES STRUCTURES EN JAVASCRIPT

1) La structure alternative (if…)

L'instruction if est la structure de test la plus basique, elle permet d'exécuter les instructions lorsqu'une condition est vérifiée, elle correspond à la structure **si….alors** en algorithmique. Sa syntaxe est la suivante.

algorithmique
Si(condition(s))
alors
liste d'instructions
finsi

Javascript
```
if (condition(s)) {
 liste d'instructions
}
```

Exemple javascript
```
if (x ==2) {
document.write ("x vaut 2") ;
}
```

2) La structure alternative (if … else…)

Elle permet en plus d'exécuter d'autres instructions en cas de non réalisation de la condition, elle correspond à la structure **si….alors….sinon** en algorithmique. La syntaxe est la suivante

algorithmique
Si(condition(s)) **alors**
liste d'instructions
Sinon
Liste d'instructions
finsi

Javascript
```
if (condition(s)) {
 liste
d'instructions}
else {
liste d'instructions
}
```

Exemple javascript
```
if (moyennne >=10) {
document.write ("admis") ;}
else{
document.write ("redouble") ;
}
```

3) Structures répétitives ou itératives (boucles)

Elles permettent d'effectuer de manière répétitive une série d'opérations jusqu'à ce qu'une condition ne soit plus réalisée. JavaScript propose plusieurs types de boucles itératives : **boucle for, boucle while et boucle do…while.**

a) La boucle while

Elle correspond à la structure algorithmique **«Tant Que».** Elle permet d'exécuter les instructions aussi longtemps que la condition est vraie. Si la condition est fausse au début, aucune instruction n'est exécutée.

Sa syntaxe est la suivante :

algorithmique
Tantque (condition(s)) **faire**
liste d'instructions
fintantque

Javascript
```
while (condition(s))
 liste d'instructions
}
```

Exemple javascript
```
<scriptlanguage="JavaScript">
    a = 0 ; somme = 0 ;{
     while (a < 25)
     a = a+1 ;
     somme = somme + a ;
}
</script>
```

b) La boucle do while

Elle correspond à la structure algorithmique **«Répéter» ou «faire…tantque».** Elle exécute les instructions aussi longtemps que la condition est vraie. Si la condition est fausse au début, les instructions seront quand même exécutées une seule fois.

Sa syntaxe est la suivante :

algorithmique
Faire
liste d'instructions
tantque (condition(s))

Javascript
Do{
liste d'instructions
} **while** (condition(s))

Exemple javascript

```
<scriptlanguage="JavaScript">
    a = 0 ; somme = 0 ;
 do {
    a = a+1 ;
    somme = somme + a ;
    } while (a < 25)
</script>
```

c) La boucle for

Elle correspond à la structure algorithmique **«Pour»** et la syntaxe est la suivante :

algorithmique
Pour (valeur initiale ; condition ; incrémentation)
liste d'instructions
finpour

Javascript

```
somme = 0 ;
for (a=0; a < 25; a = a+1 ) {
somme = somme + a;
}
```

II. LES FONCTIONS

On appelle **fonction** un ensemble d'instructions référencé par un nom et dont l'exécution se fait par simple appel de ce nom dans le corps du programme principal. Une fonction qui ne retourne aucune valeur est appelée **procédure**. Dans JavaScript, les fonctions et les procédures sont définies par le mot clé **function**.

NB : Avant d'être utilisée, une fonction ou une procédure doit être définie car pour l'appeler dans le corps du programme, il faut que le navigateur la connaisse, c.à.d. son nom, ces arguments (paramètres) et les instructions qu'elle contient. La déclaration d'une fonction ou procédure se fait en respectant la syntaxe suivante:

Function nom_fonction (parametre 1, parametre 2, ...) **{**
instruction 1
instructions 2
…..
Return resultat; /* présent si c'est une fonction */
}

Exemple : une fonction calculant le carrée d'un nombre qu'on lui passe en paramètre.

```
function carre(nombre){
result = nombre*nombre ;
return result;
}
```


ACTIVITE D'INTEGRATION 1

Soit l'algorithme suivant

1) Traduire cet algorithme en Un script javascript la première Instruction d'écriture est faite Dans une boite de dialogue et Les deux autres instructions d'écriture dans la page web

```
algorithmedecision
var note1, note2, note3, moy :réel ;
debut
ecrire ('entrer vos 3 notes') ;
lire (note1) ;
lire (note2) ;
lire (note3) ;
moy← (note1 + note2 + note3)/3 ;
si (moy>= 10) alors
ecrire ('admis') ;
sinon
ecrire (' non admis') ;
finsi
fin
```

2) Ecrire une fonction appelée Moyenne qui prend en paramètre 3 notes, calcul leur moyenne et Modifier votre script pour qu'il intègre cette fonction.
3) Soit le code suivant :

```
<html>
    <head><title>Bacc ESG</title>
      <script language="javascript">
        var noteI = 12 ;var note2 = 13;
decision;
      function moyenne(noteI, note2){
        var moyenne = (noteI+note2)/2 ;
          return moyenne ;}
          moyenne(noteI, note2) ;
          if(moyenne >=10){
            decison= "vous avez reussi au
Bacc" ;
            document.write(decision) ;}
          else{
            decision = "Echec";
            document.write(decision) ;}
      </script>
   </head>
   <body> <h1> Exo </h1> </body>
</html>
```

1. Quels langages a-t-on utilisé pour écrire ce code ?
2. Donner deux exemples de logiciels permettant d'éditer ce code.
3. Quel titre aura le document généré par ce code lorsqu'il sera interprété ?
4. Combien de variables compte ce programme ?
5. Quel sera le contenu de la variable moyenne lorsque le code sera interprété ?
6. Relever dans ce code :
   a. Une instruction d'initialisation
   b. Une instruction d'affichage
   c. Un nom de fonction
   d. Une structure de contrôle
7. Dans quel endroit de la page HTML a été inséré le code JavaScript ?

/